

MÓDULO 7: **TROUBLESHOOTING Y RESUMEN** **FINAL**

Proyecto: Análisis ETL - Entidad Financiera

Módulo final: Solución de problemas y recapitulación completa

OBJETIVO

Este módulo final proporciona:

- Soluciones a errores comunes
 - Mejores prácticas de código
 - Resumen completo del proyecto
 - Checklist de entrega
 - Consejos para presentación
-

ERRORES COMUNES Y SOLUCIONES

► **ERROR 1: ModuleNotFoundError**

Error completo:

```
ModuleNotFoundError: No module named 'pandas'
```

Causa: Biblioteca no instalada en el entorno actual.

Soluciones:

En Jupyter Local:

```
# Activar entorno virtual
cd "C:\Users\PABLO\Desktop\TECNICATURA\ABP INNOVACION"
venv\Scripts\activate

# Instalar paquete faltante
pip install pandas

# O reinstalar todo requirements.txt
pip install -r requirements.txt
```

En Google Colab:

```
# Colab ya tiene pandas, pero si falta algo:
!pip install pandas==2.1.1
```

Verificar instalación:

```
import pandas as pd
print(f'Pandas versión: {pd.__version__}')
```

► ERROR 2: FileNotFoundError al cargar CSVs

Error completo:

```
FileNotFoundError: [Errno 2] No such file or directory: 'customer_data.csv'
```

Causa: El notebook no encuentra los archivos CSV.

Diagnóstico:

```
import os

# Ver directorio actual
print(f'Directorio actual: {os.getcwd()}')

# Listar archivos en directorio
print('\nArchivos disponibles:')
for file in os.listdir('.'):
    print(f' - {file}')

# Buscar archivos CSV
print('\nBuscando archivos CSV:')
for root, dirs, files in os.walk('..'):
    for file in files:
        if file.endswith('.csv'):
            print(f' - {os.path.join(root, file)}')
```

Soluciones:

1. Verificar ruta relativa:

```
# Si el notebook está en notebooks/
df = pd.read_csv('../customer_data.csv') # Subir un nivel

# Si está en la raíz
df = pd.read_csv('customer_data.csv')    # Mismo nivel
```

1. Usar ruta absoluta (temporal):

```
import os

base_dir = r'C:\Users\PABLO\Desktop\TECNICATURA\ABP INNOVACION'
customers_path = os.path.join(base_dir, 'customer_data.csv')
df = pd.read_csv(customers_path)
```

1. Usar código adaptativo (recomendado):

```
# Ya implementado en Módulo 2
possible_paths = [
    'customer_data.csv',
    '../customer_data.csv',
    'data/customer_data.csv'
]

for path in possible_paths:
    if os.path.exists(path):
        df = pd.read_csv(path)
        print(f'✅ Archivo cargado desde: {path}')
        break
```

► ERROR 3: OSError al guardar datos_limpios.csv

Error completo:

```
OSError: [Errno 22] Invalid argument: '../datos/datos_limpios.csv'
```

Causa: Carpeta `datos/` no existe.

Solución (ya implementada en Módulo 3):

```
import os

# Crear directorio si no existe
output_dir = '../datos'
os.makedirs(output_dir, exist_ok=True)

# Ahora sí guardar
output_path = os.path.join(output_dir, 'datos_limpios.csv')
df_merged.to_csv(output_path, index=False, encoding='utf-8')
```

¿Qué hace `exist_ok=True` ?

- Si la carpeta ya existe, no da error
- Si no existe, la crea automáticamente

► ERROR 4: SettingWithCopyWarning

Warning completo:

```
SettingWithCopyWarning: A value is trying to be set on a copy of a slice from a DataFrame
```

Causa: Modificar una “vista” de un DataFrame en lugar de una copia.

Ejemplo problemático:

```
# ✗ INCORRECTO
df_subset = df[df['age'] > 30]
df_subset['age_group'] = 'Adult' # Warning aquí
```

Soluciones:

1. Usar `.copy()`:

```
# ☑ CORRECTO
df_subset = df[df['age'] > 30].copy()
df_subset['age_group'] = 'Adult' # Sin warning
```

1. Usar `.loc[]`:

```
# ☑ CORRECTO
df.loc[df['age'] > 30, 'age_group'] = 'Adult'
```

► ERROR 5: DtypeWarning al leer CSV

Warning completo:

```
DtypeWarning: Columns have mixed types. Specify dtype option on import or set low_memory=False
```

Causa: Pandas infiere tipos de datos y encuentra inconsistencias.

Solución:

```
# Especificar tipos de datos explícitamente
df = pd.read_csv('customer_data.csv', dtype={
    'customer_id': int,
    'age': float,
    'gender': str,
    'location': str,
    'membership_years': int
})

# O desactivar advertencia
df = pd.read_csv('customer_data.csv', low_memory=False)
```

► ERROR 6: KeyError al acceder columna

Error completo:

```
KeyError: 'total_sale'
```

Causa: La columna no existe en el DataFrame.

Diagnóstico:

```
# Ver todas las columnas
print(df.columns.tolist())

# Ver tipos de datos
print(df.dtypes)

# Buscar columnas similares
search_term = 'sale'
matching = [col for col in df.columns if search_term in col.lower()]
print(f'Columnas con "{search_term}": {matching}')
```

Solución:

```
# Verificar antes de acceder
if 'total_sale' in df.columns:
    print(df['total_sale'].sum())
else:
    print('⚠ Columna total_sale no existe')
    print(f'Columnas disponibles: {df.columns.tolist()}')
```

► ERROR 7: MemoryError con datasets grandes

Error completo:

```
MemoryError: Unable to allocate array with shape...
```

Causa: Dataset muy grande para la RAM disponible.

Soluciones:

1. Cargar en chunks:

```
# Procesar en lotes de 10,000 registros
chunk_size = 10000
chunks = []

for chunk in pd.read_csv('large_file.csv', chunksize=chunk_size):
    # Procesar cada chunk
    chunk_processed = chunk[chunk['age'] > 18]
    chunks.append(chunk_processed)

# Combinar todos los chunks
df = pd.concat(chunks, ignore_index=True)
```

1. Seleccionar columnas necesarias:

```
# Cargar solo columnas relevantes
cols_needed = ['customer_id', 'age', 'gender', 'location']
df = pd.read_csv('customer_data.csv', usecols=cols_needed)
```

1. Usar tipos de datos eficientes:

```
# Convertir a categorías
df['gender'] = df['gender'].astype('category')
df['location'] = df['location'].astype('category')

# Usar int32 en lugar de int64
df['customer_id'] = df['customer_id'].astype('int32')
```

► ERROR 8: UnicodeDecodeError al leer CSV

Error completo:

```
UnicodeDecodeError: 'utf-8' codec can't decode byte...
```

Causa: Encoding incorrecto del archivo.

Soluciones:

```
# Probar diferentes encodings
encodings = ['utf-8', 'latin-1', 'iso-8859-1', 'cp1252']

for encoding in encodings:
    try:
        df = pd.read_csv('file.csv', encoding=encoding)
        print(f'✅ Archivo leído con encoding: {encoding}')
        break
    except UnicodeDecodeError:
        print(f'❌ Falló con encoding: {encoding}')
```



MEJORES PRÁCTICAS

► 1. Organización de código

✗ EVITAR:

```
# Código todo junto, difícil de leer
df=pd.read_csv('file.csv')
df=df.dropna()
df['total']=df['qty']*df['price']
print(df.head())
```

☑ RECOMENDADO:

```
# Código bien estructurado con comentarios
# Cargar datos
df = pd.read_csv('file.csv')

# Limpieza
df = df.dropna()

# Crear columna calculada
df['total'] = df['quantity'] * df['price']

# Verificar resultado
print(df.head())
```

► 2. Manejo de errores

✗ EVITAR:

```
# Sin manejo de errores
df = pd.read_csv('file.csv')
```

☑ RECOMENDADO:

```
# Con manejo de errores
try:
    df = pd.read_csv('file.csv')
    print(f'✅ Archivo cargado: {len(df):,} registros')
except FileNotFoundError:
    print('❌ ERROR: Archivo no encontrado')
    print('Verifica la ruta del archivo')
except pd.errors.EmptyDataError:
    print('❌ ERROR: Archivo vacío')
except Exception as e:
    print(f'❌ ERROR inesperado: {e}')
```

► 3. Validación de datos

✅ SIEMPRE validar después de transformaciones:

```
# Antes de la transformación
print('ANTES:')
print(f'Registros: {len(df):,}')
print(f'Nulos: {df.isnull().sum().sum()}')

# Transformación
df = df.dropna()

# VALIDAR después
print('\nDESPUÉS:')
print(f'Registros: {len(df):,}')
print(f'Nulos: {df.isnull().sum().sum()}')
print(f'Registros eliminados: {len(df_original) - len(df):,}')
```

► 4. Documentación en celdas Markdown

✅ Usar Markdown para explicar:

ANÁLISIS DE VENTAS POR CATEGORÍA

En esta sección analizaremos:

- Facturación total por categoría
- Productos más vendidos
- Tendencias temporales

****Resultado esperado:**** Identificar categoría con mayor facturación (Item 4c)

► 5. Nombres de variables descriptivos

✗ EVITAR:

```
df1 = pd.read_csv('customers.csv')
df2 = pd.read_csv('sales.csv')
df3 = pd.merge(df1, df2)
```

☑ RECOMENDADO:

```
df_customers = pd.read_csv('customers.csv')
df_sales = pd.read_csv('sales.csv')
df_merged = pd.merge(df_customers, df_sales, on='customer_id')
```



CHECKLIST DE ENTREGA

► Archivos en GitHub ☒

- ☒ customer_data.csv (datos originales)
- ☒ sales_data.csv (datos originales)
- ☒ datos/datos_limpios.csv (datos procesados)
- ☒ notebooks/analisis_etl.ipynb (notebook principal)
- ☒ sql/schema.sql (esquema de BD)
- ☒ sql/consultas.sql (consultas SQL)
- ☒ sql/financional_analysis.db (base de datos)
- ☒ visualizaciones/*.png (13 gráficos)
- ☒ README.md (documentación)
- ☒ requirements.txt (dependencias)
- ☒ .gitignore (archivos ignorados)

► Contenido del notebook ☒

- ☒ Celda de configuración para Colab
- ☒ Importación de bibliotecas
- ☒ Carga de datos con verificación
- ☒ Análisis exploratorio inicial
- ☒ Limpieza de datos (nulos, duplicados)
- ☒ Transformación de fechas
- ☒ Fusión de datasets
- ☒ Creación de columnas derivadas
- ☒ Respuesta explícita a Item 4a (método de pago)
- ☒ Respuesta explícita a Item 4b (género)
- ☒ Respuesta explícita a Item 4c (facturación)
- ☒ Respuesta explícita a Item 4d (ticket promedio)
- ☒ Respuesta explícita a Item 4e (precio promedio)
- ☒ 13 visualizaciones generadas
- ☒ Implementación SQL con SQLite
- ☒ Consultas SQL ejecutadas
- ☒ Exportación de datos limpios
- ☒ Celdas con salidas (outputs) visibles

► Documentación técnica ☒

- ☒ MÓDULO 1: Configuración de entorno
- ☒ MÓDULO 2: Carga de datos
- ☒ MÓDULO 3: Limpieza y transformación
- ☒ MÓDULO 4: Análisis y visualizaciones
- ☒ MÓDULO 5: Implementación SQL
- ☒ MÓDULO 6: GitHub y Colab
- ☒ MÓDULO 7: Troubleshooting (este documento)

► Funcionalidad en Colab ☒

- ☒ Badge de Colab en README
 - ☒ URL de Colab funciona correctamente
 - ☒ Clonado automático del repositorio
 - ☒ Detección de entorno (Colab vs Local)
 - ☒ Carga de CSV adaptativa
 - ☒ Ejecución completa sin errores
 - ☒ Generación de visualizaciones
 - ☒ Consultas SQL ejecutables
-



RESUMEN DEL PROYECTO COMPLETO

► Datos procesados

Métrica	Valor
Registros originales	99,457 (clientes y ventas)
Registros limpios	99,338 (99.88%)
Registros eliminados	119 (0.12% con edad nula)
Clientes únicos	4,913
Periodo de datos	Enero 2021 - Marzo 2023
Facturación total	\$338,707,188.45
Ticket promedio	\$3,409.67

► Respuestas a los Items del TP

Item	Pregunta	Respuesta
4a	Método de pago más utilizado	Cash (44.72%)
4b	Género con más compras	Female (59.77%)
4c	Categoría con mayor facturación	Clothing (\$113.8M, 33.60%)
4d	Ticket promedio por ubicación	Ankara (\$3,459.23)
4e	Categoría con mayor precio promedio	Technology (\$3,157.35)

► Archivos generados

Tipo	Cantidad	Descripción
CSVs	3	Originales (2) + Limpio (1)
Notebooks	1	analisis_etl.ipynb (52 celdas)
SQL	3	schema.sql + consultas.sql + BD
Visualizaciones	13	Gráficos PNG (300 DPI)
Documentación	7	Módulos técnicos MD
Total archivos	27+	En repositorio GitHub

► Tecnologías dominadas

Tecnología	Uso en el proyecto
Python	Lenguaje principal (100% del código)
Pandas	ETL completo (lectura, limpieza, transformación)
NumPy	Cálculos numéricos y estadísticas
Matplotlib	13 visualizaciones (8 tipos diferentes)
SQLite3	Base de datos relacional (4 tablas + 1 vista)
SQL	17 consultas analíticas complejas
Jupyter	Entorno interactivo de análisis
Git/GitHub	Control de versiones (15+ commits)
Google Colab	Ejecución en la nube
Markdown	Documentación (2000+ líneas)

► Habilidades desarrolladas

Técnicas:

- ☒ Extracción de datos desde CSV
- ☒ Limpieza y transformación de datos
- ☒ Manejo de valores nulos y duplicados
- ☒ Fusión de datasets relacionales
- ☒ Conversión de tipos de datos
- ☒ Creación de columnas derivadas
- ☒ Análisis estadístico descriptivo

-
- ☒ Visualización de datos (8 tipos de gráficos)
 - ☒ Diseño de esquemas de bases de datos
 - ☒ Escritura de consultas SQL complejas
 - ☒ Integración Python-SQL
 - ☒ Control de versiones con Git
 - ☒ Documentación técnica profesional

Blandas:

- ☒ Resolución de problemas complejos
- ☒ Pensamiento analítico
- ☒ Atención al detalle
- ☒ Organización de proyectos
- ☒ Comunicación técnica escrita
- ☒ Trabajo autónomo
- ☒ Gestión de tiempo



CONSEJOS PARA LA PRESENTACIÓN

► 1. Preparación antes de presentar

Checklist:

- ☒ Notebook ejecutado completamente (outputs visibles)
 - ☒ Visualizaciones generadas y guardadas
 - ☒ Datos limpios exportados
 - ☒ Base de datos creada
 - ☒ README actualizado con badge de Colab
 - ☒ Repositorio pusheado a GitHub
 - ☒ Link de Colab probado y funcional
 - ☒ Commits con mensajes descriptivos
-

► 2. Estructura de presentación sugerida

1. Introducción (2 min)

- Contexto del problema
- Objetivos del análisis
- Tecnologías utilizadas

2. Proceso ETL (5 min)

- Extracción: Carga de CSVs
- Transformación: Limpieza y enriquecimiento
- Carga: Exportación y BD SQL

3. Análisis (Items 4a-4e) (5 min)

- Mostrar cada respuesta con su gráfico
- Destacar insights de negocio

4. Implementación técnica (3 min)

- Esquema de base de datos
- Consultas SQL destacadas
- Integración con Colab

5. Conclusiones (2 min)

- Hallazgos principales
- Recomendaciones de negocio
- Aprendizajes técnicos

6. Demo en vivo (3 min)

- Abrir Colab desde GitHub
- Ejecutar algunas celdas clave
- Mostrar resultados

► 3. Qué destacar

Puntos fuertes del proyecto:

1. **Calidad de datos:** 99.88% de completitud
2. **Volumen:** ~100K transacciones procesadas
3. **Automatización:** Clonado automático en Colab
4. **Documentación:** 7 módulos técnicos completos

-
5. **SQL:** 17 consultas analíticas
 6. **Visualizaciones:** 13 gráficos profesionales
 7. **Reproducibilidad:** Ejecutable en cualquier lugar
 8. **Versionado:** Control con Git/GitHub
-

► 4. Anticipar preguntas

Preguntas probables y respuestas:

P: ¿Por qué eliminaste registros con edad nula?

- R: Solo 119 registros (0.12%), pérdida mínima. Alternativa era imputar, pero eliminar garantiza datos reales.

P: ¿Por qué elegiste SQLite?

- R: Ligera, sin servidor, integración perfecta con Python, ideal para análisis de datos.

P: ¿Cómo garantizas que funcione en Colab?

- R: Celda de configuración automática que clona el repo y verifica archivos.

P: ¿Qué insights de negocio encontraste?

- R:
- Cash domina (44%), oportunidad de digitalización
- Mujeres son 60% del mercado, segmentar marketing
- Clothing es el líder (34% facturación)
- Ankara tiene mayor gasto promedio, target premium

P: ¿Qué fue lo más difícil?

- R: Configurar rutas adaptativas para Colab/Local y optimizar visualizaciones.



RECURSOS ADICIONALES

► Documentación oficial

Recurso	URL
Pandas Docs	https://pandas.pydata.org/docs/
Matplotlib Gallery	https://matplotlib.org/stable/gallery/
SQLite Tutorial	https://www.sqlitetutorial.net/
Jupyter Docs	https://jupyter.org/documentation
Git Handbook	https://guides.github.com/introduction/git-handbook/

► Comandos de referencia rápida

Pandas:

```
df.head()           # Primeras 5 filas
df.info()           # Información general
df.describe()       # Estadísticas
df.isnull().sum()   # Contar nulos
df.drop_duplicates() # Eliminar duplicados
df.groupby('col').sum() # Agrupar y sumar
pd.merge(df1, df2)   # Fusionar DataFrames
```

Git:

```
git status          # Ver estado
git add .            # Agregar todo
git commit -m "msg"  # Hacer commit
git push origin main # Subir cambios
git log --oneline    # Ver commits
```

PowerShell:

```
cd ruta           # Cambiar directorio
ls                # Listar archivos
mkdir carpeta     # Crear directorio
python script.py  # Ejecutar Python
```



CONCLUSIÓN

► Has completado exitosamente:

- ☒ **Proceso ETL completo** con 99,338 registros procesados
 - ☒ **5 respuestas** a los Items del TP con visualizaciones
 - ☒ **Base de datos SQL** con 4 tablas y 17 consultas
 - ☒ **13 visualizaciones** profesionales en alta resolución
 - ☒ **Integración GitHub-Colab** totalmente funcional
 - ☒ **7 módulos de documentación** técnica paso a paso
 - ☒ **Proyecto reproducible** ejecutable en cualquier entorno
-

► Métricas finales del proyecto:

Métrica	Valor
Líneas de código Python	1,500+
Líneas de SQL	400+
Líneas de documentación	2,500+
Commits en Git	15+
Archivos generados	27+
Tiempo de ejecución notebook	~3 min
Tiempo de desarrollo	2 semanas
Calidad del código	★★★★★

► Próximos pasos sugeridos:

1. **Revisión final:** Ejecutar notebook completo una vez más
2. **Backup:** Hacer copia local de todo el proyecto
3. **Presentación:** Preparar slides o demo
4. **Entrega:** Enviar link de Colab al profesor
5. **Celebrar:** ¡Has completado un proyecto profesional! 🎉

 ¡PROYECTO COMPLETADO!

Has desarrollado un proyecto de análisis de datos completo, desde la configuración del entorno hasta la implementación de base de datos SQL, pasando por limpieza de datos, visualizaciones y documentación técnica exhaustiva.

Habilidades demostradas:

-  Python avanzado (Pandas, NumPy, Matplotlib)
-  SQL y diseño de bases de datos
-  Análisis exploratorio de datos (EDA)
-  Visualización de datos
-  Git y control de versiones
-  Documentación técnica profesional
-  Integración con Google Colab
-  Resolución de problemas complejos

Este proyecto es una muestra sólida de tus capacidades como Data Analyst.

Documento creado: 8 de noviembre de 2025

Módulo final de: Documentación Técnica Completa - Proyecto ETL TSCDIA



CONTACTO

Para consultas sobre este proyecto:

- **Autor:** Pablo Tab
 - **GitHub:** [@Pablo-Tab](#)
 - **Repositorio:** [ABP-INNOVACION-DATOS](#)
 - **Notebook Colab:** [Abrir en Colab](#)
-

¡Éxito en tu presentación! 